Ders 19 Genişletilmiş Notu: Dinamik Programlama: Düzenleme Uzaklığı (Levenshtein) ve En Uzun Ortak Alt Dizi

Ders 19 Genişletilmiş Notu: Dinamik Programlama: Düzenleme Uzaklığı (Levenshtein) ve En Uzun Ortak Alt Dizi

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dersin Yeri ve Konusu
- 2. Düzenleme Uzaklığı (Levenshtein Uzaklığı)
- 3. Kaba Kuvvet ile MATHS ve ARTS
- 4. Dört İşlem: Kopyalama, Değiştirme, Ekleme, Silme
- 5. Dinamik Programlama Tablosu
- 6. Bağıntı ve Başlangıç Değerleri
- 7. İkinci Örnek: HONDA ve HYUNDAI
- 8. Geri İzleme ve Birden Fazla En İyi Hizalama
- 9. En Uzun Ortak Alt Dizi (LCS)
- 10. LCS Tablosu: BACDB ve BDCB
- 11. LCS Bağıntısı ve “Atlama” Fikri
- Örnek Sorular

1. Dersin Yeri ve Konusu

Ders, dönemin 10. haftasında yapıldı. Hoca, bu derste dinamik programlamayla (dynamic programming) çözülen ve uygulamada çok sık kullanılan iki temel problemi daha konuşacaklarını söyledi: düzenleme uzaklığı (edit distance) ve en uzun ortak alt dizi (longest common subsequence, LCS). Dersin başındaki ilk dakikalar dersin işleyişiyle ilgili soru-cevaba ayrıldı. Kaydın yaklaşık 00:57:40 ile 01:09:00 arası ara vermedir.

2. Düzenleme Uzaklığı (Levenshtein Uzaklığı)

Hoca konuya bir soruyla girdi: Düzenleme uzaklığı algoritmasının bir başka adı var mı? Sınıftan ses çıkmayınca cevabı kendisi verdi: Levenshtein uzaklığı (Levenshtein distance) ya da Levenshtein algoritması. Temel hedef, iki kelimenin, yani iki dizginin (string) birbirine olan mesafesini, aralarındaki farklılıklara göre ortaya koymaktır. Hocaya göre bu fikri 1965 yılında Rus bilim insanı Vladimir Levenshtein ortaya attı ve fikir bugün çok sık kullanılır hâle geldi.

Hoca tanımı İngilizce olarak yazdı: Levenshtein uzaklığı, iki dizi (sequence) arasındaki farkı ölçen bir dizgi ölçütüdür (string metric).

2.1 SUNNY Örneği: Üç Hata Türü

Hoca, bir kelimenin yanlış yazılabileceği üç farklı durumu SUNNY kelimesi üzerinden gösterdi:

- **Değiştirme (substitution, replacement):** Bir harf yanlış yazılır. Örneğin N yerine M yazılır.

- **Silme (deletion):** Bir harf eksik yazılır, aradaki bir harf atlanır.
- **Ekleme (insertion):** Fazladan bir harf yazılır.

2.2 Nerede İşe Yarar? Yazım Denetleyici

Hoca, düzenleme uzaklığının nerede işe yarayabileceğini sordu. Bir öğrenci yazım düzeltmeyi (spell correction) önerdi: Bir dizgi yanlış girildiğinde ona en yakın dizgi düzenleme uzaklığıyla bulunabilir. Hoca bunu onayladı. Yazım denetleyici (spell checker) size bir öneride (suggestion) bulunacaksa, hata yaptığınızı düşündüğü kelimenin en yakınlarını getirmesi mantıklı olandır.

Hoca ardından bunun bir program olarak hangi adımlarla yapılacağını sordu. Birlikte şu sonuca varıldı:

- İlk adım, yazılan kelimenin sözlükte (dictionary, vocabulary) olup olmadığına bakmaktır. Kelime varsa sorun yoktur.
- Kelime yoksa, ona en yakın seçenekleri önermek gerekir. Öğrencinin önerisi, en düşük düzenleme uzaklığına sahip kelimeleri önermekti.

Soru: Yazılan kelimeyle bütün sözlük arasında en küçük düzenleme uzaklığını hesaplamak pahalı olmaz mı? **Cevap:** Hoca, bu yöntemle hesaplamaya kalkarsak gerçekten böyle bir sorunla karşılaşacağımızı söyledi. İşin gerçek hayatta nasıl uygulanması gerektiğini açık bir soru olarak bıraktı. Önce yöntemi konuşup sonra bu konuya dönmeyi önerdi. Öğrenci soruyu dersin sonunda hatırlattığında da hoca, cevabı en geç bir sonraki hafta vereceğini söyledi.

> **Ek bilgi (derste anlatılmadı):** Uygulamadaki yazım denetleyiciler, büyük olasılıkla, sözlüğün her kelimesiyle tam düzenleme uzaklığı hesaplamaz. Önce aday kümesini daraltırlar. Örneğin yazılan kelimeden uzaklığı 1-2 olan bütün düzenlemeler üretilip sözlükte aranır ya da BK-ağacı (BK-tree) gibi, uzaklığa göre arama yapan veri yapıları kullanılır. Levenshtein'ın makalesi büyük olasılıkla 1965'te Rusça yayımlanmış, İngilizce çevirisi 1966'da çıkmıştır.

3. Kaba Kuvvet ile MATHS ve ARTS

Spell checker konusunu bir kenara bırakan hoca, problemi sadeleştirdi: Bize iki kelime verilsin ve aralarındaki düzenleme uzaklığı istensin. Örnek kelimeler MATHS ve ARTS idi. Hoca uzaklığı gözle tahmin etmelerini istedi. Bir öğrenci 3 dedi. Hoca bunun doğru olup olmadığını kâğıt üzerinde hizalayarak kontrol etti.

3.1 Birinci Hizalama

ARTS kelimesinin başına bir boşluk konursa iki kelime şöyle hizalanır:

```
M A T H S
- A R T S
```

- M karşısında boşluk var: bir ekleme (insert).
- A ile A aynı: değişiklik yok.
- T ile R farklı: bir değiştirme (change, replace).
- H ile T farklı: bir değiştirme.
- S ile S aynı.

Toplam üç işlem vardır, uzaklık 3 çıkar.

3.2 Başka Bir Hizalama Mümkün mü?

Hoca, kelimeleri başka türlü yerleştirme şansımız olup olmadığını sordu.

Soru: Kelimelerden birini sabit tutmamız gerekiyor mu? İşlemleri yalnız biri üzerinde mi yapabiliriz? **Cevap:** İkili bir karşılaştırma yaptığımız için hangisini sabit tuttuğumuzun önemi yoktur. Amaç aralarındaki en küçük uzaklığı bulmaktır. Bu yüzden hoca yazarken S'leri ve A'ları örtüştürmüştü. Öğrencinin eklediği gibi, boşluk başa, sona ya da araya konabilir; başka bir seçenek yoktur.

Hoca ikinci hizalamayı şöyle yazdı:

```
M A T H S
A R T - S
```

- M ile A farklı: değiştirme.
- A ile R farklı: değiştirme.
- T ile T aynı.
- H karşısında boşluk: bir ekleme ya da silme. Hocaya göre bu, hangi yönden baktığınıza bağlıdır. ARTS'tan MATHS'e geçiyorsak ARTS'ın arasına H koymuş gibi düşünürüz, bu bir eklemedir.
- S ile S aynı.

Uzaklık yine 3 çıktı.

3.3 Bu Yaklaşımın Adı

Hoca, şu ana kadar kullandıkları algoritmik yaklaşımın adını sordu. Öğrencilerin cevabı kaba kuvvet (brute force) oldu: Harfler tek tek karşılaştırılıyor ve bütün kombinasyonlara bakmak gerekiyor.

4. Dört İşlem: Kopyalama, Değiştirme, Ekleme, Silme

Hoca iki dizgiyi biçimsel olarak tanımladı: birinci dizgi $x_1 x_2 \dots x_m$, ikinci dizgi $y_1 y_2 \dots y_n$. İki dizgi arasında dört işlemden söz edilir:

1. **Kopyalama (copy):** Herhangi bir değişikliğin olmadığı durum, $x_i = y_j$.
2. **Değiştirme (change):** Daha önce yer değiştirme (replacement) ya da yerine koyma (substitute) dediğimiz işlem. Aynı sıradaki iki harfin aynı olmaması, $x_i \neq y_j$.
3. **Ekleme (insert):** MATHS ve ARTS örneğinde, bir noktaya H gelmiş ve araya yerleşmiştir.
4. **Silme (delete):** Bir harf elenmiştir.

Silmeyi göstermek için hoca önce bir örnek yazmaya başladı, sonra kafa karıştırmasın diye sildi ve başka bir kelime verdi: MAS. MAS ile ARTS eşleştirilmeye çalışıldığında S'ler denk gelir, ama aradaki T kaybolmuştur. Hoca bu durumda da uzaklığın 3 olduğunu söyledi. Python ile kontrol edildiğinde $ED(\text{MAS}, \text{ARTS}) = 3$ çıkar. En iyi hizalamalardan biri şudur (ara adım eklendi):

```
M A - - S
- A R T S
```

5. Dinamik Programlama Tablosu

Hoca, kaba kuvvet yerine dinamik programlama mantığının burada nasıl katkı sağlayacağını sordu: Nasıl bir veri yapısına ihtiyacımız var? Daha önceki bütün dinamik programlama problemlerini ek alan (space) kullanarak çözdüklerini hatırlattı: Zaman karmaşıklığını (time complexity) azalttık ama yer karmaşıklığını (space complexity) artırdık.

5.1 Tablonun Boyutu

Bir öğrenci bir matris önerdi. Önerisine göre matrisin boyutu "en uzun dizgi çarpı en uzun dizgi" kadar olacaktı. Hoca matris fikrini "fena bir mantık değil" diye karşıladı. Matrisin bir yönüne birinci dizginin, diğer yönüne ikinci dizginin harfleri yazıldı: satırlar MATHS, sütunlar ARTS.

Fakat hoca bir şeyin hesaba katılmadığını belirtti. Kelimeleri doğrudan yazınca dört ihtimalden ikisi, yani kopyalama ve değiştirme ele alınmış olur. Ekleme ve silme ihtimalleri yazılmamış olur. Bunları da işin içine katmak için **her iki tarafı da birer büyütmek** gerekir. İlk satır ve ilk sütun boşluğa (boş dizgiye) karşılık gelir. Böylece tablo $(m+1) \times (n+1)$ boyutlu olur.

> **Ders sonrası not:** Öğrencinin ilk önerdiği "en uzun dizgi çarpı kendisi" boyutu gerekli değildir. Doğrusu, hocanın da sonra kurduğu gibi, $(m+1) \times (n+1)$ boyutlu bir tablodur. MATHS ve ARTS için 6×5.

5.2 İlk Satır ve İlk Sütun

Hoca ilk hücreye ne yazılacağını sordu. Cevap 0 oldu, çünkü iki taraf da boşluktur ve aynıdır.

İlk satır için bir öğrenci önce "hepsi 1 olacak" dedi. Hoca, amacın bir kelimenin diğer kelimenin bir parçasına olan uzaklığını bulmak olduğunu hatırlattı. A'nın boşluğa uzaklığı 1'dir. AR'nin boşluğa uzaklığı 2'dir. Öğrenci bunu fark edip ilk satırın $0, 1, 2, 3, 4$ diye gideceğini söyledi.

> **Hocanın vurgusu:** Bir kelimenin boşluğa uzaklığı, o kelimenin harf sayısı kadardır.

Aynı mantıkla ilk sütun da M'den başlayarak $1, 2, 3, 4, 5$ diye ilerler. Burada M boşlukla karşılaştırılır, aradaki fark 1'dir.

5.3 M Satırı: Maliyet Nereye Eklenir?

M ile A karşılaştırıldı. Eşit değiller, demek ki bir maliyet (cost) vardır. Bir öğrenci bu hücreye 2 yazılacağını düşündü ve satırı 2, 3, 4, 5 diye sürdürdü. Hoca satırda bir sorun olup olmadığını sordu.

Bir öğrenci itiraz etti: M ile A'yı karşılaştırıyoruz, farkları 1 çıkmalı, ama tabloda 2 görünüyor. Hoca tam da buraya vurgu yapmak istediğini söyledi. Bu noktada karşılaştırılan "boşluk A" ile "boşluk M"dir. Boşluklar zaten aynıdır, tek fark M ile A'nın değişmiş olmasıdır. Bu yüzden maliyet sol üst çaprazdaki 0'a eklenmelidir: $0 + 1 = 1$. Sıfıra eklediğimiz zaman gerçek, yani en küçük uzaklığı hesaplamış oluruz. Satırın geri kalanı $2, 3, 4$ olur: boşluk M'nin boşluk AR, boşluk ART ve boşluk ARTS'a uzaklıkları.

Hoca bu arada maliyetleri netleştirdi: Değiştirmenin maliyeti 1'dir. Kopyalamanın maliyeti yoktur. Çünkü maliyet farklılığı ifade eder, kopyalamada iki harf aynıdır.

> **Hocanın vurgusu:** Önemli olan, kelimelerin alt parçaları arasındaki uzaklığı bularak iki kelimenin toplam uzaklığına erişmektir. Amaç alt problemleri çözüp o bilgileri koruyarak en sona ulaşmaktır.

5.4 A Satırı: Harfler Aynıysa

A satırında ilk karşılaştırma A ile A'dır. Öğrenci etraftaki en küçük değer olan 1'in taşınacağını söyledi. Hoca, taşımadan önce bakılması gereken bir şey olduğunu belirtti: İki harf örtüşüyor mu? Örtüşüyor, o zaman maliyet yoktur, bu bir kopyalama işlemidir. Değer 1 olur, satırın geri kalanı $2, 3, 4$ olarak devam eder.

> **Ders sonrası not:** A satırındaki konuşmada eşitlik durumunda çaprazın devre dışı bırakıldığı izlenimi doğabilir; doğrusu, hocanın birkaç dakika sonra toparladığı gibi, harfler aynıysa sol üst çaprazdaki değer maliyetsiz taşınır. A ile A hücresindeki 1 de çaprazdaki $ED[1, 0] = 1$'den gelir.

5.5 T Satırı ve Kuralın Toparlanması

T satırında bir öğrenci, T ile A farklı olduğu için üstteki değere 1 ekleyip 2 yazdı. Sonra T ile R farklı olduğu için yine üsttekine 1 ekleyip 3 yazdı. Hoca "acaba?" diye sordu. Başka bir öğrenci, farklılık durumunda çapraza da baktıklarını hatırlattı.

Hoca kuralı şöyle toparladı:

- Eşitlik durumu varsa çaprazdaki değer taşınır.
- Farklılık durumu varsa maliyet oluşturan üç ihtimal vardır: değiştirme, ekleme ve silme. Üçünün her birine 1 eklenir ve en küçük değer hangisinden geliyorsa yeni kutuya o yazılır.

Bu kuralla T satırı şöyle dolar:

- T ile A farklı: komşuların en küçüğü 1, sonuç 2.
- T ile R farklı: komşular (çapraz 1, üst 2, sol 2), en küçüğü 1, sonuç 2. Öğrencinin 3 cevabı bu yüzden yanlıştı.
- T ile T aynı: çaprazdaki 2 maliyetsiz taşınır, sonuç 2.
- T ile S farklı: en küçük komşu 2, sonuç 3.

Sonraki iki satırı hoca kendisi doldurdu. H, A, R, T ve S'nin hiçbiriyle aynı değildir, her hücreye en küçük komşu artı 1 yazılır: $3, 3, 3, 3$. S satırında S, A, R ve T'den farklıdır: $4, 4, 4$. S ile S aynıdır, çaprazdaki 3 taşınır. Tablonun tamamı aşağıdadır (Python ile doğrulandı):

	ε	A	R	T	S
ε	0	1	2	3	4
M	1	1	2	3	4
A	2	1	2	3	4
T	3	2	2	2	3
H	4	3	3	3	3
S	5	4	4	4	3

En küçük uzaklık tablonun sağ alt köşesindedir: $ED(\text{MATHS}, \text{ARTS}) = 3$. Bu, kaba kuvvetle bulunan sonuçla aynıdır.

6. Bağıntı ve Başlangıç Değerleri

Hoca dört durumu tablodaki yönlerle özetledi:

- **Kopyalama:** Sol üst çaprazdaki değer olduğu gibi aşağıya aktarılır. Değişiklik olmamıştır, maliyet yoktur.
- **Değiştirme:** Çaprazdaki değerin üzerine 1 eklenerek aktarılır.
- **Ekleme:** Yukarıdaki değerin üzerine 1 eklenerek aktarılır.
- **Silme:** Soldaki değerin üzerine 1 eklenerek aktarılır.

Kopyalama olmadığında bu üç seçenekten en küçüğü bir sonraki kutuya yazılır, çünkü amaç en küçük uzaklığı bulmaktır.

6.1 Dört Durumun Formülü

Hoca durumları öğrencilerle birlikte formülleştirdi:

- **Durum 1 (kopyalama):** $ED[i, j] = ED[i-1, j-1]$. Kopyalamada maliyet eklenmez.
- **Durum 2 (değiştirme):** Yine çaprazdan erişilir: $ED[i, j] = ED[i-1, j-1] + \text{cost}$. Maliyet her seferinde 1'dir.
- **Durum 3 (ekleme):** Sütun sabit, bir önceki satırdan, yani yukarıdan gelinir: $ED[i, j] = ED[i-1, j] + \text{cost}$.
- **Durum 4 (silme):** Satır sabit, soldan gelinir: $ED[i, j] = ED[i, j-1] + \text{cost}$.

Hoca ekleme ve silmeyi, harflerin birbirinin üzerine nasıl oturduğunu gösteren küçük hizalamalarla da çizdi. Eklemede x tarafına bir harf eklenmiş gibi düşünülür ve yukarıdaki hücreye bakılır.

Hangisinin ekleme, hangisinin silme sayıldığı, hocanın da birkaç kez söylediği gibi, dönüşümün hangi yönde düşünüldüğüne bağlıdır. Uzaklığın değeri bu adlandırmadan etkilenmez.

6.2 Bağıntı

Bağıntıda (recurrence relation) bir "eğer" vardır: x_i, y_j'ye eşit mi? Bir öğrenci eşitlik durumunda 0 yazılacağını önerdi. Hoca, bir sonraki kutunun değerinin sol üst köşedeki kutudan belirlendiğini, bağıntı kurmak için değerin bir öncekinden taşınması gerektiğini söyledi. Eşit olmama durumu için bir başka öğrenci, maliyetin 1 olacağını ve komşulardan hangisi küçükse ona ekleneceğini söyledi. Hoca bunu üç seçeneğin en küçüğü olarak yazdı:

$$ED[i, j] = \begin{cases} ED[i-1, j-1] & x_i = y_j \text{ ise} \\ 1 + \min\{ED[i-1, j-1],\ ED[i-1, j],\ ED[i, j-1]\} & \text{aksi hâlde} \end{cases}$$

Hocanın vurgusu: Bir sonraki değeri belirlerken bakılabilecek üç yer vardır. Ama önce harflerin örtüşüp örtüşmediğine bakılır. Örtüşme varsa maliyet yoktur, köşegen doğrudan aşağı taşınır. Yoksa silme, ekleme ya da değiştirme olmuştur. Bunların

hepsi olmuş olabilir, ama yola en küçük olanla devam edilir. Optimal çözüme ulaşmak için alt problemlerin en iyi çözümlerini bulmamız lazım.

6.3 Başlangıç Değerleri

Matris ilklendirilirken (initialize) ilk sütun ve ilk satır şöyle doldurulur:

$$ED[i, 0] = i, \qquad ED[0, j] = j$$

Bunlar boşluğa olan uzaklıklardır, yani yalnız ekleme ya da silmeyle elde edilen değerlerdir.

> **Ders sonrası not:** Harfler aynıyken çapraz değeri doğrudan taşımak güvenlidir. Komşu hücreler arasındaki fark en fazla 1 olduğu için çapraz değer, diğer iki seçenekten (komşu + 1) hiçbir zaman büyük olmaz. Bu yüzden eşitlik durumunda üst ve sol komşulara ayrıca bakmaya gerek yoktur.
>
> **Ek bilgi (derste anlatılmadı):** Tablo $(m + 1)(n + 1)$ hücreden oluşur ve her hücre sabit sürede doldurulur. Bu yüzden düzenleme uzaklığı algoritmasının zaman ve yer karmaşıklığı $\Theta(mn)$'dir. Yalnız uzaklık isteniyorsa iki satır tutularak yer $\Theta(\min(m, n))$'e indirilebilir, ama o zaman geri izleme yapılamaz.

7. İkinci Örnek: HONDA ve HYUNDAI

Aradan sonra hoca, konunun daha rahat oturması için ikinci bir örnek çözdü: HONDA ile HYUNDAI arasındaki düzenleme uzaklığı. Satırlara HONDA, sütunlara HYUNDAI yazıldı. İlk satır ve ilk sütun boşluğa karşılık gelir ve ilklendirilir.

Satır satır ilerleyiş şöyle oldu:

- **H satırı:** H ile H aynıdır, bu bir kopyalamadır. Çaprazdaki 0 taşınır. Sonraki harfler farklı olduğu için satır $1, 2, 3, 4, 5, 6$ diye sürer.
- **O satırı:** O ile H hücresine ne yazılacağını hoca sordu. Bir öğrenci 1 dedi. Hoca bu değeri yorumlamalarını istedi: H'ler aynıdır, bu noktada bir ekleme ya da silme gerçekleşmiştir. Hangisi olduğu kelimeyi hangi tarafa yazdığınıza bağlıdır. O ile Y farklıdır, değer 1 olur. Satır $2, 3, 4, 5, 6$ diye devam eder.
- **N satırı:** N ile H, Y ve U hücreleri 2 olur. N ile N aynıdır, köşegenden 2 taşınır. N ile D 3, N ile A 4, N ile I 5 olur.
- **D satırı:** D; H, Y, U ve N ile aynı değildir, hücreler 3 olur. D ile D aynıdır, köşegenden 2 taşınır. Sonra 3 ve 4 gelir.
- **A satırı:** Hoca bu satırı alt kelimeler üzerinden yorumladı. HONDA, H ile karşılaşınca fark 4'tür. HY, HYU ve HYUN ile karşılaşınca da fark 4'tür. HYUND ile fark 3 olur, çünkü D'ler eşleşir. HYUNDA ile A'lar eşleşir ve fark 2 olur. En son I gelir. I, A ile eşit olmadığı için en küçük komşuya 1 eklenir ve sonuç 3 olur.

Tablonun tamamı (Python ile doğrulandı):

	ε	H	Y	U	N	D	A	I
ε	0	1	2	3	4	5	6	7
H	1	0	1	2	3	4	5	6
O	2	1	1	2	3	4	5	6
N	3	2	2	2	2	3	4	5
D	4	3	3	3	3	2	3	4
A	5	4	4	4	4	3	2	3

Sonuç: $ED(\text{HONDA}, \text{HYUNDAI}) = 3$.

8. Geri İzleme ve Birden Fazla En İyi Hizalama

Hoca uzaklığı hesapladıktan sonra, bu uzaklığın hangi işlemlerle elde edilmiş olabileceğini de ekranda gösterdi. İki farklı hizalama vardı. Python ile bulunan iki en iyi hizalama şunlardır:

```
H - O N D A -       H O - N D A -
H Y U N D A I       H Y U N D A I
```

- **Birinci hizalama:** H'ler eşleşir. Y eklenir. O ile U arasında değiştirme olur. N, D ve A eşleşir. Sonda I eklenir.
- **İkinci hizalama:** H'ler eşleşir. O ile Y arasında değiştirme olur. U eklenir. N, D ve A eşleşir. Sonda I eklenir.

İkisinde de bir değiştirme ve iki ekleme vardır, uzaklık 3'tür. Hoca da her iki hizalamada bir değiştirme ile iki ekleme ya da silme olduğunu söyledi.

8.1 Eksik Olan Ne?

Hoca bir soru sordu: Optimal değeri bulduk, ama matriste neyi yapmadık? Bir öğrenci cevap verdi: Farkların nerede olduğuna hiç bakmadık, yalnız kaç tane fark olduğunu bulduk. Hoca bunu onayladı. Peki matrisi kullanarak hangi yoldan geldiğimizi ve farkların nerede olduğunu nasıl bulabiliriz?

8.2 Sondan Başa Doğru Gitmek

Başka bir öğrenci sondan geriye doğru gitmeyi önerdi. Sağ alt köşedeki 3'ten başlanır. Komşuların en küçüğü 2'dir, demek ki 3'e 2'den gelinmiştir. Sonra A ile A aynıdır. Öğrenci burada "2'den gelmiş diyemeyiz, yukarıdaki ya da soldaki 3'ten gelmiştir" diyerek bir an kafası karıştı.

Başka bir öğrenci daha düzenli bir yöntem önerdi: Harflere bakmak yerine sayılara bakılabilir. Çaprazdaki 2 ile bulunulan 2 aynıysa, orada bir değişiklik yoktur. Yani o harf için değiştirme, ekleme ya da silme yapılmamıştır. D'lerde ve N'lerde de çapraz giderek aynı şey anlaşılır. Öğrencinin algoritması şöyle özetlenebilir:

- Bulunulan hücrenin komşularına bakılır. Komşulardan biri daha küçükse, oradan bir işlemle gelinmiştir.
- Komşuların hiçbiri kendisinden küçük değilse ve harfler aynıysa, çapraza atlanır. O harf için bir işlem yapılmamıştır.

Hoca bu noktada şunu gösterdi: Bir noktaya geldiğinizde, örneğin 2'nin üzerine ya solundan ya da çaprazından gelmiş olabilirsiniz. İki farklı durum vardır, sonra iki yol birleşir. H'ler çakışır. Aşağıya inildiğinde bir tarafta Y, bir tarafta O vardır. U tarafından gidildiği varsayılırsa N'ler, D'ler ve A'lar eşittir, son noktada bir ekleme vardır. Öbür yoldan gidilirse diğer hizalama elde edilir.

Hocanın geri izleme kuralı:

- Geriye giderken kendisinden küçük olan değere doğru ilerlenir.
- Kendisinden küçük değer yoksa, ya çapraza gidilir ya da kendisine eşit olan değer tercih edilir.
- Kendisine eşit iki değer varsa, iki farklı noktadan gelmiş olma olasılığı vardır. Bu iki farklı en iyi hizalama demektir.

  Hocanın vurgusu: Böylece kaç tane farklı durum olabileceğini de hesaplama şansımız vardır. İki kelimenin birbirine en az uzaklıkta olmasını sağlayan farklı hizalamaları değerlendirebiliriz. Bu da önemlidir. MATHS ve ARTS örneğinde olduğu gibi, nereye boşluk geldiği, hangi harfin silindiği ya da hangi noktaya ekleme yapıldığı farklı olsa da sonuç aynı çıkabilir.

9. En Uzun Ortak Alt Dizi (LCS)

Hoca ikinci problem olarak en uzun ortak alt dizi problemine geçti. LCS genellikle iki farklı örüntünün (pattern) birbiri içinde bulunma durumunu bulmak için kullanılan bir yaklaşımdır. Özellikle biyoinformatikte (bioinformatics) iki DNA diziliminin birbirine benzerliğini hesaplamak için kullanılır. DNA dizilimlerinde A, C, G ve T harfleri kullanılır.

9.1 DNA Örneği: ATCTGAT ve TGCATA

Hoca iki dizi verdi: X = ATCTGAT, Y = TGCATA. Gözle bakarak $LCS(X, Y)$'nin ne olduğunu sordu.

Bir öğrenci önce 3, sonra 5 dedi ve harfleri birebir karşılaştırmaya başladı. Hoca burada iki dizginin birebir eşleşmesiyle ilgilenmediklerini, en uzun örüntüyü atlamalı bir şekilde aradıklarını söyledi. Örnek olarak TCTA'yı gösterdi.

Hocanın açıklamasına göre, uzun bir DNA dizilimi düşünüldüğünde araya başka değerler girmiş olabilir. Bunlar alt diziyi (subsequence) bozan değerler gibi görünse de, TCTA'nın sırası korunarak iki dizinin içinde de geçtiği görülür.

- Bir öğrenci CAT'i önerdi. CAT iki dizide de geçer, ama uzunluğu 3'tür. Hoca, 4'lük bir çözüm zaten bulunduğu için en az 4 olması gerektiğini, daha büyüğü varsa onun peşinde olduklarını söyledi.
- Bir başka öğrenci TGAT'ı önerdi. Hoca kontrol etti ve bunun ikinci bir seçenek olduğunu söyledi. Düzenleme uzaklığında olduğu gibi burada da birden fazla en iyi çözüm olabilir.

Python ile kontrol edildiğinde LCS uzunluğu 4'tür ve uzunluğu 4 olan üç farklı ortak alt dizi vardır: TCTA, TGAT ve TCAT.

> **Ders sonrası not:** Derste TCTA ve TGAT bulundu. Bunlara ek olarak TCAT de uzunluğu 4 olan bir en uzun ortak alt dizidir.

9.2 Alt Dizi Ne Demek? KOZALAK Örneği

Hoca alt dizi kavramını bir kelimeyle gösterdi. Kelimenin içinden KAL, KOL, KOLA ve KALK çıkarılabilir. Harfler ardışık olmak zorunda değildir, ama sırası korunur. Buna karşılık AZ bir alt dizi değildir, çünkü kelimede Z, A'dan önce gelir. Python ile kontrol edildiğinde KAL, KOL, KOLA ve KALK, KOZALAK'ın alt dizisidir, AZ ise değildir.

9.3 Biçimsel Tanım

$X = x_1 x_2 \dots x_m$ ve $Y = y_1 y_2 \dots y_n$ olsun. Bir ortak alt dizi $S = s_1 s_2 \dots s_k$, her iki dizide de artan indislerle geçen dizidir. Yani X için $i_1 < i_2 < \cdots < i_k$ olacak şekilde indisler vardır ve $s_t = x_{i_t}$'dir. Aynısı Y için de geçerlidir. LCS, bu koşulu sağlayan en uzun S'dir.

Hoca bu problemde de bir matris kullanacaklarını ve dinamik programlama mantığıyla ilerleyeceklerini söyledi. Tablo C adını alır ve son değer $C[m, n]$ hücresinde bulunur.

10. LCS Tablosu: BACDB ve BDCB

Örnek olarak X = BACDB ve Y = BDCB verildi. Satırlara BACDB, sütunlara BDCB yazıldı.

10.1 Kaç Durum Var?

Hoca önce kaç durumun konuşulacağını sordu. Bir öğrenci iki durum olduğunu söyledi: eşit ya da eşit değil. Eşit olmama durumunun alt durumları var mı diye sorulunca öğrenci, ekleme ve silme bizi ilgilendirmediği için alt durum olmadığını söyledi.

Hoca bunun doğru olduğunu, ama bir detayı vurgulamak gerektiğini belirtti: Eşit olmama durumunda hangi taraftaki örüntüde atladığımızı belirtmemiz lazım. X tarafında mı, Y tarafında mı atlıyoruz? Çünkü bir sonraki turda atladığımız harfi öbür dizinin içinde bulma şansımız vardır. Bu yüzden temelde üç durumdan söz edilir:

- Harflerin eşit olma durumu.
- X'te atlama (skip x).
- Y'de atlama (skip y).

10.2 Tablonun Doldurulması

İlk satır ve ilk sütun için bir öğrenci, henüz eşleşme olmadığı için 0 yazılacağını söyledi. Hoca bunu onayladı: Boşlukla bir harfin eşleşme durumu söz konusu değildir.

B satırı: B ile B eşleşir, 1 yazılır. Düzenleme uzaklığında en küçük uzaklığı arıyorduk, burada en uzun eşleşmeyi arıyoruz. B ile D eşleşmez, ama "heybemizde" bir tane 1 vardır, yine 1 yazılır. B ile C de 1 olur. Satırın son hücresinde öğrencinin kafası karıştı: B ile B yine eşleşiyor, ama öncesinde de B ile B eşleşmişti. Hoca kâğıt üzerinde bakmalarını istedi: Bir tarafta B, bir tarafta BDCB var, cevap 1 olmalı. Bu 1, sol üst çaprazdaki 0'a 1 eklenerek gelir. Hocanın ifadesiyle, eşleşme varsa bir önceki çapraz yerden buraya ulaşmış olmamız gerekir.

A satırı: A, B ile eşleşmez. Öğrenci, B ile B önceden eşleştiği için 1 yazdı. Hoca bunun hangi seçim olduğunu sordu. Öğrenci üstten geldiğini söyledi. Hoca, öğrencinin aslında büyük olanı, yani maksimumu aldığını belirtti: İki taraftan 1 ve 0 gelir. Daha önce yakalanan eşleşme hangi tarafta büyükse o alınır, çünkü en büyük eşleşme isteniyor. A satırının tamamı 1 olur.

C satırı: C ile B ve C ile D eşleşmez, 1 taşınır. C ile C eşleşir, çaprazdaki 1 artırılır, 2 olur. C ile B eşleşmez, 2 taşınır.

D satırı: D ile B eşleşmez, üstteki 1 taşınır. D ile D eşleşir, çaprazdaki 1 artırılır, 2 olur. D ile C eşleşmez, maksimum 2 taşınır. Öğrencinin belirttiği gibi burada üstteki de soldaki de 2'dir, ikisi de taşınabilir. Hoca bunun, iki farklı noktadan kesişen iki farklı örüntü olabileceğini gösterdiğini söyledi. D ile B eşleşmez, yine 2.

B satırı (son): B ile B eşleşir, çaprazdaki 0 artırılır, 1 olur. B ile D eşleşmez, 2 taşınır. B ile C eşleşmez, 2 taşınır. B ile B eşleşir, çaprazdaki 2 artırılır, 3 olur.

Tablo (Python ile doğrulandı):

	ε	B	D	C	B
ε	0	0	0	0	0
B	0	1	1	1	1
A	0	1	1	1	1
C	0	1	1	2	2
D	0	1	2	2	2
B	0	1	2	2	3

En uzun ortak alt dizinin uzunluğu 3'tür. Hoca bunların neler olabileceğini sordu. Öğrencilerin cevabı BCB ve BDB oldu. Hoca başka seçenek olmadığını söyledi. Python ile kontrol edildiğinde uzunluğu 3 olan ortak alt diziler gerçekten yalnız BCB ve BDB'dir.

10.3 Alt Problemler Üzerinden Yorum

Hoca tabloyu alt diziler üzerinden bir kez daha okudu. B ile B'nin eşleşmesinde LCS 1'dir. BA'da A'nın başka bir şeyle eşleşmesi yoktur. BAC, önce B ile, sonra BD ile karşılaştırılır, değer 1'de kalır. BAC ile BDC karşılaştırılınca değer 2 olur. Çünkü bir önceki BA ile BD'nin en çok kesişen örüntüsü B'dir ve bu bilgi çaprazdaki hücrede saklıdır. Buna C'lerin eşleşmesi eklenir.

BACD ile BDCB'de son harfler kesişmez. Bu durumda önceki kesişimlere bakılır, değer hâlâ 2'dir. En sonda iki diziye de B eklenir. B'ler kesiştiği için, geri kalanların içindeki ikili kesişime 1 eklenir. Geri kalanlarda iki kesişim vardır, BC ya da BD. Toplamda 3'e ulaşılır.

11. LCS Bağıntısı ve "Atlama" Fikri

Hoca kuralları yazdı. Önce başlangıç: $i = 0$ ise $C[0, j] = 0$. Hoca bunun ayrıca kontrol edilmesine gerek olmadığını, görmeleri için yazdığını söyledi. Aynısı $j = 0$ için de geçerlidir.

Eşitlik durumunda bir öğrenci, sol üst çapraza 1 eklendiğini söyledi. Eşit olmama durumunda iki komşunun maksimumu alınır. Bağıntı şöyledir:

$$C[i,j] = \begin{cases} 0 & i = 0 \text{ ya da } j = 0 \text{ ise} \\ C[i-1, j-1] + 1 & i, j > 0 \text{ ve } x_i = y_j \text{ ise} \\ \max\{C[i-1, j],\ C[i, j-1]\} & i, j > 0 \text{ ve } x_i \neq y_j \text{ ise} \end{cases}$$

Hoca kuralı sözlü olarak da özetledi: Kesişime bakmadan önce harflerin aynı olup olmadığına bakılır. Aynıysa köşegendeki değere 1 eklenir. Aynı değilse ya birinci örüntüde ya da ikinci örüntüde bir atlama olmuştur. Hangisiyle devam edileceğine, o ana kadar en büyük eşleşme hangi taraftaysa ona göre karar verilir.

11.1 Atlama Neden Önemli?

Hoca, atlama konusunu neden gündeme getirdiğini sordu ve yorum bekledi. Cevap gelmeyince kendisi açıkladı. BACDB ve BDCB örneğinde, hangi örüntüde ilerleyeceğinize karar vermeniz gerekir. Yukarıdaki örüntüyle ilerleyecekseniz skip x, aşağıdakiyle ilerleyecekseniz skip y yapmanız gerekir ki eşleşmeyi buna göre bulasınız.

> **Hocanın vurgusu:** Bunu kodlamaya döktüğünüzde yalnızca "eşit değildir" üzerinden kodlayamazsınız. Mutlaka X üzerinde $i + 1$ ya da Y üzerinde $j + 1$ yapmanız gerekir. Bu yüzden hangi tarafta ilerlemeniz gerektiğini de belirlemeniz gerekir.

Hoca, LCS'nin sözde kodunu (pseudocode) bir sonraki hafta yazacağını söyledi. O zamana kadar skip x ya da skip y'nin nerede uygulanması gerektiğini düşünmelerini istedi.

> **Ek bilgi (derste anlatılmadı):** LCS tablosu da $(m + 1)(n + 1)$ hücreden oluşur ve her hücre sabit sürede doldurulur, bu yüzden zaman ve yer karmaşıklığı $\Theta(mn)$'dir. Alt dizinin kendisi, sağ alt köşeden geriye doğru gidilerek bulunur: Harfler eşitse çapraza gidilir ve harf alt diziye eklenir, değilse büyük değeri veren komşuya (üst ya da sol) gidilir. İki komşu eşitse iki yol da denenerek bütün en uzun ortak alt diziler bulunabilir.

Örnek Sorular

Bu dersin video kaydı ve kareleri yoktur. Aşağıdaki sorular derste sözlü olarak sorulan sorulardır.

1. (derste sözlü, 00:08:07) Düzenleme uzaklığı (edit distance) algoritmasının bir başka adı nedir?

> **Çözüm:** Levenshtein uzaklığı (Levenshtein distance) ya da Levenshtein algoritması. Hocaya göre fikir 1965'te Vladimir Levenshtein tarafından ortaya atıldı.

2. (derste sözlü, 00:12:27) Düzenleme uzaklığı nerede işimize yarar? Bir yazım denetleyici bunu hangi adımlarla kullanır?

> **Çözüm:** Yazım düzeltmede (spell correction). Önce yazılan kelimenin sözlükte olup olmadığına bakılır. Varsa sorun yoktur. Yoksa sözlükte ona en yakın, yani düzenleme uzaklığı en küçük kelimeler önerilir. Bütün sözlükle karşılaştırmanın maliyeti sorusunu hoca sonraki derse bıraktı. Bu konuda 2.2'deki ek bilgi bloğuna bakılabilir.

3. (derste sözlü, 00:16:19) MATHS ile ARTS arasındaki düzenleme uzaklığı kaçtır? Gözle söyleyebilir misiniz? Başka türlü hizalama mümkün mü?

> **Çözüm:** 3. `MATHS / -ARTS` hizalamasında bir ekleme ve iki değiştirme, `MATHS / ART-S` hizalamasında iki değiştirme ve bir ekleme vardır. İkisi de 3 verir. Dinamik programlama tablosu da (Bölüm 5.5) sağ alt köşede 3 verir.

4. (derste sözlü, 00:20:31) Uzaklığı hesaplamak için bütün hizalamaları kâğıt üzerinde denemeye hangi algoritmik yaklaşım denir?

> **Çözüm:** Kaba kuvvet (brute force). Bütün kombinasyonlara bakmak gerekir.

5. (derste sözlü, 00:24:24) Kaba kuvvet yerine dinamik programlama kullanmak için nasıl bir veri yapısına ihtiyacımız var?

Çözüm: Bir matris. Bir yönüne birinci dizginin, diğer yönüne ikinci dizginin harfleri yazılır. Ekleme ve silme durumlarını da kapsamak için iki taraf da birer büyütülür: $(m+1) \times (n+1)$ boyutlu tablo. İlk satır ve ilk sütun boşluğa karşılık gelir.

6. (derste sözlü, 00:31:09) Tablonun ilk hücresine, ilk satırına ve ilk sütununa ne yazılır?

Çözüm: İlk hücreye 0 yazılır, çünkü iki taraf da boşluktur. İlk satır $0, 1, 2, \ldots, n$ ve ilk sütun $0, 1, 2, \ldots, m$ diye gider, çünkü bir kelimenin boşluğa uzaklığı harf sayısı kadardır: $ED[0, j] = j$, $ED[i, 0] = i$.

7. (derste sözlü, 00:36:00) Değiştirmenin maliyeti 1'dir. Kopyalamanın maliyeti var mı?

Çözüm: Yoktur. Maliyet farklılığı ifade eder. Kopyalamada iki harf aynıdır, çapraz-daki değer olduğu gibi taşınır.

8. (derste sözlü, 01:11:18) HONDA ile HYUNDAI tablosunda O ile H'nin kesiştiği hücreye ne yazılır? Bu değer hangi işlemi gösterir?

Çözüm: 1. H'ler zaten eşleşmiştir. Bu hücrede fazladan bir harf vardır, yani bir ekleme ya da silme gerçekleşmiştir. Hangisi olduğu, dönüşümün hangi yönde düşü-nüldüğüne bağlıdır.

9. (derste sözlü, 01:16:40) HONDA ile HYUNDAI için optimal değeri bulduk. Matriste neyi yapmadık?

Çözüm: Farkların nerede olduğuna, hangi işlemlerin yapıldığına bakmadık. Bu, sağ alt köşeden geriye doğru izlenerek bulunur. Eşit iki komşu varsa iki farklı en iyi hizalama vardır. Bu örnekte `H-ONDA- / HYUNDAI` ve `HO-NDA- / HYUNDAI` hizalamaları çıkar.

10. (derste sözlü, 01:26:33) X = ATCTGAT, Y = TGCATA için $LCS(X, Y)$ nedir?

Çözüm: Uzunluk 4. Derste TCTA ve TGAT bulundu. CAT ortak alt dizidir, ama uzun-luğu 3 olduğu için en uzun değildir. Python ile bulunan uzunluğu 4 olan bütün ortak alt diziler: TCAT, TCTA, TGAT.

11. (derste sözlü, 01:33:10) En uzun ortak alt dizi probleminde kaç farklı durum değerlen-dirilir?

Çözüm: Temelde eşit olma ve eşit olmama durumları vardır. Ama eşit olmama duru-munda hangi dizide atlandığı belirtilmelidir. Bu yüzden üç durum konuşulur: eşitlik, X'te atlama (skip x) ve Y'de atlama (skip y).

12. (derste sözlü, 01:42:08) BACDB ile BDCB için en uzun ortak alt dizinin uzunluğu 3 bulundu. Bu alt diziler nelerdir?

Çözüm: BCB ve BDB. Python ile kontrol edildiğinde uzunluğu 3 olan başka ortak alt dizi yoktur.

13. (derste sözlü, 01:50:00) LCS'de "atlama" (skip x / skip y) konusu neden önemlidir?

Çözüm: Hocanın cevabına göre, kodlarken yalnızca "eşit değil" koşuluyla ilerlene-mez. Hangi dizide ilerleneceğine karar verilmeli, yani ya X üzerinde $i + 1$ ya da Y üzerinde $j + 1$ yapılmalıdır. Atlanan harf, öbür dizide sonradan eşleşebilir.

Dersin Toparlaması

Bu derste dinamik programlamayla çözülen iki dizgi problemi görüldü. Düzenleme uzaklığında, iki dizgi arasındaki en az ekleme, silme ve değiştirme sayısı $(m+1)\times(n+1)$ boyutlu bir tabloyla hesaplandı. Tablonun ilk satırı ve sütunu boşluğa olan uzaklıkları tutar. Harfler aynıysa çap-raz değer maliyetsiz taşınır, farklıysa üç komşunun en küçüğüne 1 eklenir. MATHS ile ARTS ve HONDA ile HYUNDAI örneklerinde uzaklık 3 bulundu. Uzaklığın yanı sıra hangi işlemle-rin yapıldığı da tablodan geri izlenerek çıkarıldı ve aynı uzaklığı veren birden fazla hizalama olabileceği görüldü.

En uzun ortak alt dizide ise iki dizide sırası korunarak geçen en uzun alt dizi arandı. Harfler eşitse çapraz değere 1 eklenir, değilse üst ve sol komşulardan büyük olanı taşınır. BACDB ile BDCB için uzunluk 3 bulundu, alt diziler BCB ve BDB idi. Hoca, eşit olmama durumunda

hangi dizide atlandığının (skip x ya da skip y) önemine dikkat çekti. Bir sonraki hafta LCS'nin sözde kodu yazılacak ve yazım denetleyicide bütün sözlükle karşılaştırma maliyeti sorusuna dönülecek.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.